Slide 4: Computation & Verification

Speaker: Ehtesham Faraz

</> mpi_matmul.c — ikj compute loop

```
/* ikj loop order: maximises cache reuse of B rows */
for (int i = 0; i < my_rows; i++) {
    for (int k = 0; k < n; k++) {
        double temp = local_A[i * n + k]; /* scalar hoist */
        for (int j = 0; j < n; j++) {
            local_C[i * n + j] += temp * B[k * n + j];
        }
    }
}

/* Collect result slices on Rank 0 for verification */
if (rank == 0) {
    double checksum = 0;
    for (long i = 0; i < (long)n * n; i++)
        checksum += C[i];
    printf("Checksum: %f\n", checksum);
}
```

📈 **OBSERVATION**

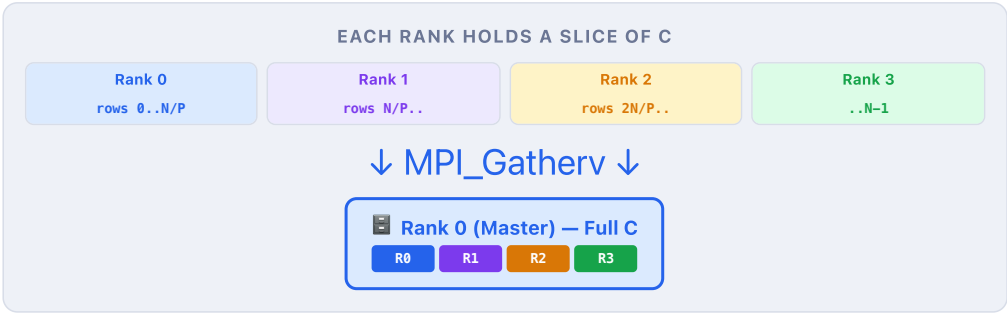
ikj order keeps B rows in L1/L2 cache across the inner j-loop, greatly reducing cache misses vs. naive ijk.

⚠️ **LIMITATION**

Full B replica (N×N per node) limits scalability. For very large N, a Cannon's algorithm avoids this overhead.

📩 MPI_Gatherv — Collect to Rank 0

Each rank sends its computed slice of C. Only Rank 0 allocates the full result matrix.



✅ Verification Methodology

- **Checksum Calculation**
  Rank 0 sums all N^2 elements of C using `double` arithmetic. Compared against a sequential baseline for correctness.

- **Validation Status**
  Checksum matches sequential reference across all tested configurations.
  ✓ PASSED — Checksum matches sequential baseline

COMPUTE / NODE	COMMUNICATION
$O(N^3/P)$	$O(N^2)$
ideal linear speedup	one-time Allgatherv